

Übungsblatt 11

Punkte:

--

 / 20

Fortgeschrittene Algorithmen (Wintersemester 2022/23)

Bearbeitet von: **Matthias Janßen, Raphael Thelen, Tillmann**

Faust

February 6, 2026

Aufgabe 1

a)

Seien aktuell $S_1.size() + S_2.size() = n$. Wenn S_2 mindestens ein Element enthält dauert die Operation $\Theta(1)$. Wenn S_2 leer ist müssen zuerst alle Elemente auf S_2 verschoben werden und anschließend das oberste entfernt werden. Da die Operation im worst case maximal n Werte bewegt und jede Push- und Pop-Operation für eine Bewegung in $\Theta(1)$ läuft ist die worst-case Laufzeit $\Theta(n)$.

b)

Per Buchhaltermethode: Wir geben bei jeder Enqueue-Operation 2 Kosteneinheiten zusätzlich zu den Push-Kosten hinzu. Die Gesamtkosten sind damit $\Theta(1)$ und jedes Element hat 2 Kosten als Budget. Wenn nun Dequeue aufgerufen wird liegt entweder ein Element auf S_2 und wir haben Kosten 1, oder wir können die Kosten für die Push- und Pop-Operation jeden Elements auf S_1 durch die 2 zurückgelegten Kosten bezahlen und müssen nur noch 1 für die Letzte Pop-Operation Zahlen. Damit kostet jede Dequeue-Operation amortisiert nur noch $\Theta(1)$.

Aufgabe 2

a)

```
inc() {
  for all i in 0...k-1 {
    if A[i] == 0 {
      A[i] = 1
      return
    } else {
      if i == k-1 {
        set all to 0
        return
      }
    }
  }
}
```

```

    A[i] = 0
  }
}
return A
}

```

b)

Im worst case sind alle Bits 1 und es muss jedes Bit auf 0 gesetzt werden. Somit sind die worst-case Kosten $\mathcal{O}(k)$.

c)

Wir zahlen für das setzen eines Bits von 0 auf 1 2 €. 1 € wird für den eigentlichen Setzvorgang verwendet, der andere € wird als Kredit gespeichert. Die tatsächlichen Kosten für das Setzen eines Bits auf 1 sind $\mathcal{O}(1)$, die amortisierten Kosten liegen bei $\mathcal{O}(1) + 1$. Somit bleibt jedem Bit 1 Euro Kredit welcher verwendet werden kann, um das Bit später wieder auf 0 zu setzen. Jedes Bit zahlt die Operation selbst. Der Kontostand wird also nie negativ. Somit sind die amortisierten Kosten pro Inkrementierung $\Theta(1)$.

Aufgabe 1

a)

Seien aktuell $S_1.size() + S_2.size() = n$. Wenn S_2 mindestens ein Element enthält dauert die Operation $\Theta(1)$. Wenn S_2 leer ist müssen zuerst alle Elemente auf S_2 verschoben werden und anschließend das oberste entfernt werden. Da die Operation im worst case maximal n werte bewegt und jede Push- und Pop-Operation für eine Bewegung in $\Theta(1)$ läuft ist die worst-case Laufzeit $\Theta(n)$.

b)

Per Buchhaltermethode: Wir geben bei jeder Enqueue-Operation 2 Kosteneinheiten zusätzlich zu den Push-Kosten hinzu. Die Gesamtkosten sind damit $\Theta(1)$ und jedes Element hat 2 Kosten als Budget. Wenn nun Dequeue aufgerufen wird liegt entweder ein Element auf S_2 und wir haben Kosten 1, oder wir können die Kosten für die Push- und Pop-Operation jeden Elements auf S_1 durch die 2 zurückgelegten Kosten bezahlen und müssen nur noch 1 für die Letzte Pop-Operation Zahlen. Damit kostet jede Dequeue-Operation amortisiert nur noch $\Theta(1)$.

Aufgabe 2

a)

```

inc() {
    for all i in 0...k-1 do
        if A[i] == 0
            A[i] = 1
            return
        else
            if i == k-1
                set all to 0
                return
            A[i] = 0
    return A
}

```

b) Im worst case sind alle Bits 1 und es muss jedes Bit auf 0 gesetzt werden. Somit sind die worst-case Kosten $\mathcal{O}(k)$.

c) Wir zahlen für das setzen eines Bits von 0 auf 1 2 €. 1 € wird für den eigentlichen Setzvorgang verwendet, der andere € wird als Kredit gespeichert. Die tatsächlichen Kosten für das Setzen eines Bits auf 1 sind $\mathcal{O}(1)$, die amortisierten Kosten liegen bei $\mathcal{O}(1) + 1$. Somit bleibt jedem Bit 1 Euro Kredit welcher verwendet werden kann, um das Bit später wieder auf 0 zu setzen. Jedes Bit zahlt die Operation selbst. Der Kontostand wird also nie negativ. Somit sind die amortisierten Kosten pro Inkrementierung $\Theta(1)$.

Aufgabe 3

a)

1. **Konstruktion:** Nacheinander n INSERT-Operationen in einen leeren Fibonacci-Heap. $\Rightarrow$ Wurzelliste besteht aus n Bäumen mit Grad 0.
2. **EXTRACTMIN():**
 - Minimum entfernen, es verbleiben $n - 1$ Knoten.
 - CLEANUP-Operation.
 - Konsolidierung durchläuft die gesamte Wurzelliste, echten Kosten sind proportional zur Anzahl der Wurzeln + maximaler Grad, hier jeweils Null.
 - Mit $n - 1$ Wurzeln betragen die echten Kosten

$$T = O(n - 1 + D(n)) = O(n - 1 + 0) = \Omega(n)$$

b)

1. **Konstruktion:** Baum mit Wurzel w und Grad k .

2. Die Wurzel w besitzt k Kinder $c_1, c_2, \dots, c_k$. Wir entfernen für jedes Kind c_i alle Kinder mittels DECREASE-KEY und anschließendem CUT.
3. Da wir bei DECREASE-KEY nicht markieren, wird c_i nicht wie üblich von w abgeschnitten sondern bleibt teil des Baums.
4. Sobald alle Enkel entfernt wurden, sind $c_1, \dots, c_k$ Blätter mit Grad 0.
5. Wurzel w hat weiterhin grad k , die Gesamtzahl an Knoten beträgt

$$1 + k = \Theta(k)$$

Aufgabe 4